Project Milestone P2

Analysis and High-Level Design

LMS AI Agent

Team	Group 1, section .010
Document version	1.0 (baseline for P2; grows from the P1 baseline, v0.5)
Date	November 08, 2026
Repository URL	https://github.com/azghier1/Ai-Agent
Git tag for this milestone	p2 (points at the commit that contains this exact PDF)
Submitted on Canvas by	John Bramlet

Team member	NetID	GitHub username
John Bramlet	dal222463	JohnDavid-B
Erik Sandoval	eas220006	ErikS11
Omor Igiehon	ooi180001	
Daniel Habib	dxh230020	danielrhabib
Ahmad Zghier	dal901977	azghier1
Zaid Abusamra	dal304926	zabusamra1

0 Revision History, Contribution Record and Changes Since P1

Version	Date	Author	Summary of change
0.1	Oct 18, 2026	John Bramlet	Set up the shared document from the course template; completed sections 0 and 1 of P1
0.2	Oct 18, 2026	Zaid Abusamra, Omor Igiehon	Completed section 2: context diagram, stakeholders, user characteristics, constraints, assumptions and feasibility
0.3	Oct 18, 2026	Erik Sandoval	Completed section 3: FR-01 to FR-08, QR-01 to QR-04, CON-01 and CON-02, use-case diagram, UC-01 and UC-02, traceability matrix
0.4	Oct 18, 2026	Daniel Habib, Ahmad Zghier	Completed sections 4 and 5 of P1: team roles, plan to P2, risks and open questions
0.5	Oct 18, 2026	John Bramlet	P1 baseline as submitted (issued as "1.0" at P1; renumbered under CH-09 so that the P2 baseline is 1.0)
0.6	Nov 7, 2026	John Bramlet	Working prototype committed: agent, Moodle adapter, Moodle deployment, sample course, chat bubble; team onboarding guide and P2 work plan
0.7	Nov 8, 2026	All members	Sections 2 to 9 added or updated through pull requests (see 0.1)
1.0	Nov 8, 2026	John Bramlet	P2 baseline: analysis and high-level design added; sections 0, 1 and 10 to 12 updated

0.1 Individual contribution record (P2)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
John Bramlet	Sections 0, 1 (update), 5.1 class table, 5.2 UC-01 (scenario table and sequence diagram), 5.3 cross-checks, 8 (spike S-01: script, 100-request measurement, README, section text, with Zaid Abusamra as co-author), 10, 11 and 12; final PDF assembly script and tag `p2`. Before P2 drafting: the working prototype the design describes (agent, Moodle LMS adapter, Moodle deployment and sample-course seed, chat page and Moodle chat bubble), the team onboarding guide and the P2 work plan	Prototype on `main`: `b5c18f7`, `f019be3`, `c5ee6e5`, `347886c`. PR #1 (onboarding guide, work plan): `9b3bad1`, `4db0b68`, `124579b`, `b908084`. PR #6 (sections 0, 1, 10 to 12, PDF build): `1c9db9c`, `150046e`, `d35c61c`, `8af4582`. PR #8 (section 8, spike): `baed82d`, `c275bb4`, `ecdc987`. PR #11 (sections 5.1 table, 5.2 UC-01, 5.3)	PR #4 (Erik), PR #3 (Daniel), PR #9 (Daniel), PR #10 (Zaid)

Team member	Sections and artifacts this person produced for P2	Evidence in the repository (commit hashes, PR numbers)	Reviews given (PR numbers)
Erik Sandoval	Section 3 update (QR-02 reworded, CON-01 changed to Moodle, use-case IDs reconciled to UC-01 to UC-04, use-case diagram) and section 4 (use cases selected, expanded, self-reviewed)	PR #4: `07e984f`, `d8b9049`	None
Omor Igiehon	Domain model diagram for 5.1 (draw.io source and image, committed to his fork and merged by John). Reviewed Daniel's section 7 and 5.2 UC-02. The 5.1 class table, 5.2 UC-01 and 5.3 assigned to him were not done; John completed them	Fork OmiIgi/Ai-Agent, branch docs/p2-omor-domainModel, merged in PR #11: `e150b01`, `61d01db`, `959a139`, `980fda8`	PR #2 (Daniel), PR #3 (Daniel)
Daniel Habib	Section 5.2 for UC-02 (non-trivial steps, scenario table, analysis sequence diagram and its source) and section 7 (ADR-01 to ADR-05, trade-off summary)	PR #2: `5152e68`, `957ab26`, `874853c`, `d1dc743`, `ddaeb1b`, `7a09149`, `02838bb`, `0209b82`, `b46b878`. PR #3: `dce6a7d`, `7c18fb6`, `13fb907`, `00031c3`, `19f1888`, `aa28138`	PR #5 (Ahmad): changes requested, then approved; PR #8 and PR #11 (John)
Ahmad Zghier	Section 6 (QAS-01 to QAS-03, style and alternatives, component view, deployment view, analysis-to-component mapping, design principles) with diagram sources	PR #5 (merged as `4d85643`): `9aec1ea`, `e6b95e0`, `6d5b491`, `35dbf54`, `bc798e2`, `64a375a`, `949ef0c`, `7ce4bbb`, `575ec50`, `56c5ea0`	PR #7 (Zaid): changes requested, then approved
Zaid Abusamra	Section 2 update (feasibility verdicts tied to repository evidence, survey instrument) and section 9 (traceability matrix for all 14 requirements, gap analysis); co-author of section 8 (spike S-01)	PR #7: `20f021f`, `468bc7f`, `ffbd50d`; co-author on `ecdc987` (PR #8)	PR #6 (John): changes requested, then approved

0.2 Changes since the P1 baseline

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-01	Repository in use: every P2 section reaches `main` through a reviewed pull request; 0.1 cites commit hashes and PR numbers	P1 feedback (C4, C5): the repository had no commits, so conventions and contributions could not be verified	10.4 describes the layout and the PR rule; the contribution record is checkable	Nov 8

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-02	LMS changed from Canvas to self-hosted Moodle 5.3 (CON-01 reworded; 1.1 and 1.3 updated; Q1 closed)	Canvas Lite, the free hosted Canvas, blocks personal API tokens for every account, so the agent could not connect. Moodle's web-service API works from a self-hosted instance	ADR-02; R1 closed; sample data loaded into Moodle (R3); the LMS adapter is Moodle-specific	Nov 7
CH-03	Feasibility verdicts in section 2 tied to evidence	P1 feedback (C1): verdicts were conditional with no checkable evidence. The prototype now makes working calls against a self-hosted LMS API (commits `b5c18f7`, `c5ee6e5`)	Technical feasibility rests on the prototype; performance feasibility waits on spike S-01 (section 8)	Nov 8
CH-04	Stakeholder needs in 2.2: a five-question student survey replaces assumption as the method	P1 feedback (C1): needs were stated as assumptions	Survey results are tracked in Q4 until reported	Nov 8
CH-05	Use-case IDs reconciled to UC-01 to UC-04 (3.4); the P1 matrix's UC-05 reference corrected to UC-04	P1 feedback (C3): matrix referred to UC-03 and UC-05 while the diagram showed UC-01 to UC-04	Section 4 and the matrix in 9.1 use the same four IDs	Nov 8
CH-06	QR-02 reworded with task, population, threshold, time limit and sample size	P1 feedback (C3): QR-02 could not be tested	QR-02 is now verifiable by a 10-student usability test	Nov 8
CH-07	Contribution record cites real repository evidence	P1 feedback: the contribution table said repository links were "to be added"	0.1	Nov 8
CH-08	Use-case detail moved from 3.4 to section 4; the P1 matrix in 3.5 replaced by a pointer to 9.1	P2 template structure	No content lost; IDs preserved	Nov 8
CH-09	Document version numbering: P1 baseline renumbered 0.5	P1 was issued as "1.0"; the P2 baseline must be 1.0	Revision history above	Nov 8
CH-10	Campus sign-in constraint (2.4) is **not implemented** in the prototype: the chat service does not authenticate users and reads Moodle with one configured test-student token. Per-student sign-in through LTI 1.3 is planned for P3	Prototype finding; raised in review of PR #6	ADR-05; QAS-02 not yet demonstrable; R7	Nov 8

Change ID	What changed (IDs affected)	Why (P1 feedback, new finding, spike result)	Knock-on effect	Agreed on
CH-11	Plan, risks and open questions updated (10.2, 10.3, 11): R1 closed, R6 to R8 added, Q1 to Q3 closed, Q4 to Q6 added	Analysis and design findings, platform change	Sections 10 and 11	Nov 8

1 Introduction

1.1 Purpose of this document and of the system

This document is the P2 analysis and high-level design baseline (version 1.0) for the LMS AI Agent, written by Group 1 (Section 010) for the CS 3354 instructor and teaching assistants, and for the team as the reference that P3 implementation and testing will trace back to. It carries forward the P1 problem, scope and requirements under change control (section 0.2) and adds use-case analysis, analysis models, the architecture and its rationale.

The LMS AI Agent is a study assistant for students that works on top of a learning management system. A student's course information is spread across modules, pages, announcements and assignment pages in every course they take, and the LMS gives no way to ask a plain-language question about that material or to see what is due at a glance. The agent lets a signed-in student ask questions answered only from their own enrolled courses' material, see their assignments and what is due soon, and practice with generated questions.

For development and testing the team runs its own instance of Moodle 5.3, an open-source LMS, loaded with sample course data. P1 planned on Canvas; the free hosted Canvas blocks the API access the agent needs, so the team moved to Moodle (CH-02, ADR-02). The problem belongs to students; the office that runs a university's LMS has a stake in keeping any such agent inside the access a student already has.

1.2 Scope

In scope: we will build this	Out of scope: we will not, and why
Answering students' natural-language questions using only the material in courses they are enrolled in (FR-01 to FR-03)	Answers drawn from the open web or from courses the student is not enrolled in: we cannot vouch for accuracy, and it would reach past the student's own access
Retrieving a student's assignments and submission status from the LMS, sorted by due date, with the next seven days flagged (FR-04 to FR-06)	Submitting, editing or deleting any work in the LMS on a student's behalf: the agent only reads; writing raises academic-integrity and approval issues
Generating practice questions from course material (FR-08)	Completing graded assignments or quizzes for students: this would be academic dishonesty, not study help

In scope: we will build this	Out of scope: we will not, and why
Clear error messages when LMS data cannot be retrieved (FR-07), with each student identified through their own LMS sign-in	Instructor and administrator features such as grading, course setup or analytics, and support for more than one LMS: we have one term and one user class to serve well

Unchanged since v0.5 except for the LMS named in the last row (CH-02, CH-10).

1.3 Definitions, acronyms and abbreviations

Term or abbreviation	Meaning as used in this project
LMS	Learning Management System: the platform a university uses to deliver courses
Moodle	Open-source LMS. The team self-hosts Moodle 5.3 for development and testing (CON-01)
Canvas	The LMS used at UT Dallas, built by Instructure. Named in P1; replaced by Moodle for this project (CH-02)
Moodle web services	Moodle's interface for other software to read course data on behalf of a signed-in user
AI agent	Our system: software that takes a student's request, retrieves the data it needs from the LMS, and uses a large language model to produce an answer or list
LLM	Large language model: the AI model that writes the agent's answers and practice questions
Course material	Content a student can already see in an enrolled course: modules, pages, announcements and assignment descriptions. Excludes other students' work and anything hidden from the student
Grounded answer	An answer built only from course material, or an explicit statement that the material does not contain enough information (FR-03)
Practice question	A question generated from course material for self-study; it is never submitted or graded in the LMS (FR-08)
Verified student	A student who has signed in to the LMS and is enrolled in the course being asked about (FR-02, QR-03)
LTI 1.3	Learning Tools Interoperability: the standard way an LMS launches an external tool and tells it who the signed-in user is. Planned for P3 (ADR-05)
QAS	Quality-attribute scenario: a measurable quality goal used to drive the architecture (section 6.1)
ADR	Architecture decision record: a design decision with its alternatives and cost (section 7)
Spike	A small, time-boxed experiment that answers a design question with a measured result (section 8)
FERPA	Family Educational Rights and Privacy Act: the U.S. law protecting student education records such as grades and submissions [R4]
SSO	Single Sign-On: the campus login service students already use

1.4 References

[R1] ISO/IEC/IEEE 29148:2018, Systems and software engineering, Life cycle processes, Requirements engineering. [R2] Moodle HQ, Moodle 5.3 web services and external functions documentation. [R3] UT Dallas OIT, campus single sign-on and Duo two-factor authentication guidance. [R4] Family Educational Rights and Privacy Act (FERPA), 20 U.S.C. § 1232g; 34 CFR Part 99. [R5] CS 3354 Fall 2026 syllabus, P1 project brief and P2 project description. [R6] Anthropic, Claude API documentation (Messages API, tool use). [R7] 1EdTech, Learning Tools Interoperability (LTI) 1.3 specification.

2 Overall Description

2.2 Stakeholders and evidence

Primary stakeholder: a student using Moodle to track coursework, understand course materials, and practice concepts. Other stakeholders include instructors whose course policies must be respected and administrators responsible for LMS access and data protection.

The prototype supports asking about courses, assignments, syllabus, pages, modules and announcements. These are prototype capabilities, not proof that students requested them. P1 feedback identified stakeholder needs as assumptions. A five-question survey is proposed to evaluate those assumptions; no collected responses are available in this contribution.

Proposed survey (anonymous, voluntary, with no names or student IDs): (1) How often do you miss or nearly miss LMS deadlines? Never/Rarely/Sometimes/Often. (2) Which LMS tasks are hardest? Select all: find deadlines, find assignment instructions, understand syllabus policies, review notes, practice concepts, other. (3) How useful would an AI assistant embedded in the LMS be? 1–5. (4) Which concern matters most? Accuracy, privacy, speed, academic integrity, accessibility, other. (5) Would you use a read-only study assistant that does not complete graded work? Yes/Maybe/No.

Evidence status: the survey is not completed in the available evidence; no responses or aggregate results have been supplied in this contribution. The stakeholder needs above remain assumptions supported by the project's intended scope, not validated student findings. No sample size, response counts, percentages or usefulness score is claimed. Validation would require actual anonymized aggregate results and a documented collection method. A convenience sample of classmates would be exploratory, not representative of all students. The P1 stakeholder-evidence issue remains open.

2.6 Feasibility summary

Technical feasibility — implementation evidence exists; independent runtime validation and production readiness remain unverified. Prototype commit b5c18f7 adds the agent and Moodle adapter; commit 347886c adds the web chat and Moodle bubble. The Docker configuration specifies Moodle 5.3 and PostgreSQL. The adapter implements course and assignment retrieval and the agent exposes seven

read-only tools. These are code and configuration evidence; this contribution does not provide an independently reproduced end-to-end demo log.

Integration feasibility — conditional. The team's onboarding record reports that its Canvas Lite environment did not provide the needed token access, motivating the move to self-hosted Moodle. This records the team's environment-specific finding, not a claim about every Canvas deployment. The prototype relies on one configured student's token; it does not provide identity isolation for multiple real students. LTI 1.3 and a way to obtain each student's authorized Moodle access are planned, not implemented.

Performance feasibility — open. QR-01's target (95% of responses within 5 seconds under 20 concurrent users) is not verified by this contribution. John's Section 8 S-01 experiment is the designated evidence source. Its actual results must determine the verdict; no timing or pass/fail outcome is claimed here.

Privacy/security feasibility — conditional. Student question/context is sent to the external Anthropic Claude API over HTTPS. The current shared-token approach can expose one test student's data to anyone reaching the chat server. Production deployment requires per-user authentication, authorization and privacy review; the current prototype is for test data only.

Operational/economic feasibility — conditional. Moodle and the chat server require hosting and maintenance, and Claude API usage incurs cost. Actual usage costs and deployment assumptions are not documented in this contribution, so a production cost verdict is not established.

Verdict: the repository provides an implemented prototype and documented setup path using synthetic/test data. This supports a limited technical feasibility assessment, while stakeholder validation, independently reproduced integration, production identity/privacy, performance and operating cost remain unresolved. Production deployment feasibility is not yet established.

Additional implementation evidence: student-token Moodle REST adapter and mock data interface, chat endpoint and shared-token/session limits, and agent prompt, model loop and read-only tools. Missing evidence is explicit: survey results, a reproducible live integration record, S-01 measurements and usage-cost records have not been supplied here. Section 9 maps these limitations to requirements and follow-up work.

3 Specific Requirements

The requirements from the P1 baseline are retained and updated to reflect the team's selection of Moodle 5.3, the professor's feedback on QR-02, and the reconciliation of the use-case identifiers. Original requirement IDs have been preserved.

3.1 Functional Requirements

ID	Requirement	Priority	Source	Acceptance Criterion
FR-01	The system shall accept natural-language questions submitted by students.	MUST	Students	Submit 10 questions of varying lengths and verify that each is received completely without truncation.
FR-02	The system shall retrieve course information from the connected LMS only for courses in which the verified student is enrolled.	MUST	Students	Compare retrieved course information against the enrolled student's accessible course records.
FR-03	The system shall generate answers to student questions based only on retrieved course information.	MUST	Students	Submit at least 8 course-related questions and verify the answers against accessible course materials or confirm that insufficient information is reported.
FR-04	The system shall retrieve the student's assignments and their submission status from each enrolled course.	MUST	Students	Compare retrieved assignments and submission statuses with the LMS records for the test student.
FR-05	The system shall list assignments in ascending order of due date.	MUST	Students	Provide assignments with different due dates and verify chronological ordering.
FR-06	The system shall identify assignments due within the next seven calendar days.	MUST	Students	Test assignments due inside and outside the seven-day period and verify that only those within the period are identified.
FR-07	The system shall display an error message when requested LMS information cannot be retrieved.	MUST	Students	Simulate an LMS retrieval failure and verify that the student receives an understandable error message.
FR-08	The system shall generate practice questions based on course materials.	SHOULD	Students	Select a course, request five practice questions, and verify that each relates to the selected course's material.

The functional requirement IDs and priorities remain unchanged from P1.

3.2 Quality Requirements

ID	Requirement	Priority	Source	Verification
QR-01	The system shall respond to at least 95% of student queries within 5 seconds while supporting 20 concurrent users.	MUST	Students	Submit 100 queries with 20 concurrent simulated users and measure the response times.
QR-02	The system shall enable at least 90% of first-time student users to locate their upcoming assignments within 60 seconds, without external assistance, during a usability test involving at least 10 students.	SHOULD	Students	Ask 10 first-time users to locate upcoming assignments. Measure completion time and verify that at least 9 complete the task within 60 seconds without assistance.

ID	Requirement	Priority	Source	Verification
QR-03	The system shall deny 100% of attempts by unauthenticated or unauthorized users to access protected course records, measured over at least 20 test attempts.	MUST	Students	Attempt unauthorized access using separate test accounts and verify that every request is denied.
QR-04	The system shall achieve at least 95% factual accuracy on a predefined evaluation set of 100 questions answerable from accessible LMS course materials.	MUST	Students	Compare generated responses against an instructor-reviewed answer key and verify at least 95 correct responses.

QR-02 Correction

In P1, QR-02 did not clearly define when the student was expected to complete the assignment-finding task. The revised requirement defines:

- Task: Locate upcoming assignments.
- Population: First-time student users.
- Success threshold: At least 90%.
- Time limit: 60 seconds.
- Sample size: At least 10 students.
- Condition: No external assistance.

The requirement is now measurable and can be evaluated through a repeatable usability test.

3.3 Constraint Requirements

ID	Constraint	Source	Rationale
CON-01	The system shall integrate with the team's selected self-hosted Moodle 5.3 LMS for development and testing.	Project scope and team platform decision	Moodle provides the course-data access required by the prototype and fits the team's available development environment.
CON-02	The system shall operate within the development environment and resource limits established for the CS 3354 course project.	Course project instructions and team resource agreement	The implementation must remain feasible within the team's resources and academic schedule.

Platform Change: Canvas to Moodle

The P1 proposal originally planned integration with Canvas. During development, the team found that its Canvas Lite environment did not provide the required API token access.

The team selected self-hosted Moodle 5.3 instead.

The current prototype demonstrates access to course information, assignments, announcements, and course pages using Moodle.

This change updates CON-01 without changing the project's central purpose: providing a read-only AI study assistant for students.

3.4 Initial Use Cases

The P1 feedback identified inconsistent use-case numbering between the diagram and traceability matrix.

The consolidated P2 use-case list is:

Use Case	Name	Primary Actor	Related Requirements
UC-01	Ask Course-Related Questions	Student	FR-01, FR-02, FR-03, FR-07
UC-02	Organize Assignments	Student	FR-02, FR-04, FR-05, FR-07
UC-03	View Upcoming Deadlines	Student	FR-02, FR-04, FR-06, FR-07
UC-04	Generate Practice Questions	Student	FR-02, FR-03, FR-08, FR-07

Use-Case Numbering Reconciliation

Figure 3.4 — AI Study Assistant Use-Case Diagram

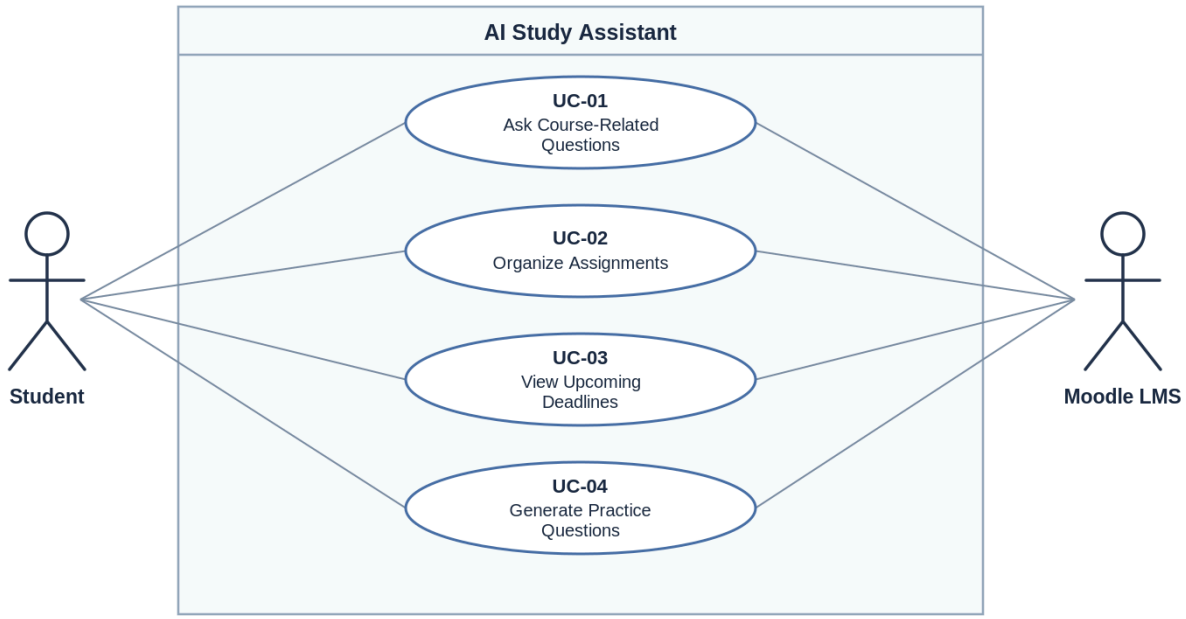


Figure 3.4. Student interactions with the AI Study Assistant and Moodle LMS.

UC-01 and UC-02 retain their original identifiers and names.

UC-03 represents the student goal of identifying upcoming assignment deadlines within seven days.

UC-04 represents generating practice questions, consistent with the use-case diagram described in the P1 feedback.

The P1 traceability matrix incorrectly referenced UC-05 for practice-question generation. That reference is corrected to UC-04. No separate UC-05 actor goal was established in the P1 documentation.

The revised use-case model therefore uses UC-01 through UC-04 consistently.

Scope and Implementation Status

UC-01 and UC-02 represent core system functions and are supported by the current prototype, although some specified behaviors remain incomplete.

Source references for UC-01, automated cross-course deadline organization, and practice-question generation are planned requirements rather than fully verified prototype features.

All four use cases describe the expected system behavior for subsequent development and evaluation.

Detailed analysis of selected use cases appears in Section 4.

3.5 Requirements Traceability

The extended requirements traceability matrix is maintained in Section 9.1 of the P2 document.

That matrix connects functional, quality, and constraint requirements to use cases, analysis classes, architectural components, and supporting design decisions.

The P1 traceability inconsistencies involving UC-03, UC-04, and UC-05 are resolved through the consolidated use-case list in Section 3.4.

4 Use-Case Analysis

This section expands three student-facing use cases from Section 3.4. The scenarios describe required user-visible behavior rather than internal implementation. UC-01 and UC-02 were introduced in P1; UC-04 covers practice-question generation.

4.1 Use Cases Selected for Analysis

Use case	Name	Why it was selected	Non-trivial steps
UC-01	Ask Course-Related Questions	This is the primary purpose of the assistant. It must provide grounded answers, protect course access, and handle unavailable information.	Steps 2, 4, 6

Use case	Name	Why it was selected	Non-trivial steps
UC-02	Organize Assignments	Students need to understand their assignments, deadlines, and submission statuses. Missing dates and unavailable records introduce important alternatives.	Steps 2, 4
UC-04	Generate Practice Questions	This supports the project's study-assistance goal and requires the system to generate relevant questions without completing graded work.	Steps 2, 4

These use cases were selected for their importance to students, requirement coverage, and the risks associated with missing or inaccessible course information.

4.2 Expanded Use Cases

UC-01 — Ask Course-Related Questions

Field	Description
Primary actor	Student
Supporting actors and external systems	Learning Management System (Moodle)
TUCBW	The student wants an answer to a question about an enrolled course.
TUCEW	The student receives an answer with its supporting course reference, or an understandable explanation of why the request could not be completed.
Preconditions	The student is authenticated and has access to at least one enrolled course.
Postconditions	On success, the student has received a course-grounded answer and a supporting source reference. On failure, the student has been informed why an answer cannot be provided.
Traces to requirements	FR-01, FR-02, FR-03, FR-07, QR-03, QR-04, CON-01

Main Success Scenario

Actor input/action	System response
—	(0) The system presents the student's available courses and indicates that course-related questions can be asked.
(1) The student selects a course.	(2) The system confirms the selected course and invites the student to ask a question.
(3) The student submits a question about the course material.	(4) The system presents an answer supported by accessible course information and identifies the source material used.

Actor input/action	System response
(5) The student asks to examine the supporting material.	(6) The system identifies the relevant course page, assignment, or other source and confirms that the supporting material has been located.

Alternate and Exception Flows

Flow	At step	Condition	What happens	Ends or rejoins at
A1	4	The course material does not contain sufficient information to answer the question.	The system explains that the available material does not support an answer and does not invent one.	Ends with an insufficient-information notification.
A2	4	The requested course information cannot be retrieved.	The system informs the student that the information is temporarily unavailable and that the request was not completed.	Ends with a failure notification.
A3	2	The selected course is not accessible to the student.	The system denies access to that course, explains the restriction, and offers the student's available courses.	Rejoins at step 1.
A4	6	The supporting material is no longer accessible.	The system explains that the source cannot be displayed and does not substitute unverified information.	Ends with a source-unavailable notification.

The supporting reference is part of the required behavior. Source-reference functionality remains a known implementation gap for P3.

UC-02 — Organize Assignments

Field	Description
Primary actor	Student
Supporting actors and external systems	Learning Management System (Moodle)
TUCBW	The student wants to review assignments organized by their due dates.
TUCEW	The student sees assignments ordered by due date and can identify an assignment's deadline and submission status, or receives a clear explanation of why the information is unavailable.
Preconditions	The student is authenticated and has access to at least one enrolled course.
Postconditions	On success, the student sees the available assignments in chronological order with their relevant information. On failure, the student receives an explanation.
Traces to requirements	FR-02, FR-04, FR-05, FR-07, QR-02, QR-03, CON-01

Main Success Scenario

Actor input/action	System response
—	(0) The system presents the student's available courses and indicates that assignment information can be requested.
(1) The student requests an organized list of assignments from their enrolled courses.	(2) The system presents the available assignments in ascending order of due date, including course names, due dates, and submission statuses.
(3) The student requests more information about a particular assignment.	(4) The system presents the assignment's available instructions, due date, and submission status, and confirms that the assignment details have been provided.

Alternate and Exception Flows

Flow	At step	Condition	What happens	Ends or rejoins at
A1	2	The student has no available assignments.	The system explains that no assignments were found in the student's accessible courses.	Ends with an empty-result acknowledgment.
A2	2	One or more assignments have no due date.	The system displays those assignments after all dated assignments and identifies them as having no specified deadline.	Rejoins at step 2.
A3	2	Assignment information cannot be retrieved.	The system explains that the requested information is temporarily unavailable and does not present incomplete information as a complete list.	Ends with a retrieval-failure notification.
A4	4	The requested assignment is no longer accessible.	The system informs the student that the assignment details are unavailable and offers the available assignment list.	Rejoins at step 1.

UC-02 describes the required cross-course organization behavior. The prototype can retrieve assignment information, but fully automated cross-course organization remains a development gap.

UC-04 — Generate Practice Questions

Field	Description
Primary actor	Student
Supporting actors and external systems	Learning Management System (Moodle)
TUCBW	The student wants to practice concepts from an enrolled course.

Field	Description
TUCEW	The student receives five course-related practice questions for self-study, or an explanation of why they could not be generated.
Preconditions	The student is authenticated and has access to at least one enrolled course.
Postconditions	On success, the student has five practice questions based on accessible course material. The student's course records and graded work remain unchanged. On failure, the student receives a clear explanation.
Traces to requirements	FR-02, FR-03, FR-07, FR-08, QR-03, QR-04, CON-01

Main Success Scenario

Actor input/action	System response
—	(0) The system presents the student's available courses and indicates that study questions can be requested.
(1) The student selects a course for practice.	(2) The system confirms the selected course and indicates that questions can be generated from its available material.
(3) The student requests five practice questions based on the selected course.	(4) The system presents five questions relevant to the course material and confirms that they are intended for self-study rather than graded submission.

Alternate and Exception Flows

Flow	At step	Condition	What happens	Ends or rejoins at
A1	4	The course contains insufficient accessible material to create five relevant questions.	The system explains that there is not enough available course information and does not invent unsupported course content.	Ends with an insufficient-material notification.
A2	4	Course information cannot be retrieved.	The system reports that the course material is currently unavailable and that practice questions could not be generated.	Ends with a retrieval-failure notification.
A3	2	The selected course is not accessible to the student.	The system explains that the course cannot be accessed and offers the student's available courses.	Rejoins at step 1.
A4	3	The student requests questions that would directly complete graded work.	The system declines to complete graded work and offers self-study questions about the relevant concepts instead.	Rejoins at step 3 if the student accepts the alternative.

This use case specifies expected P3 behavior. The current prototype's tutoring and quiz capabilities do not by themselves establish that the complete practice-question workflow has been verified.

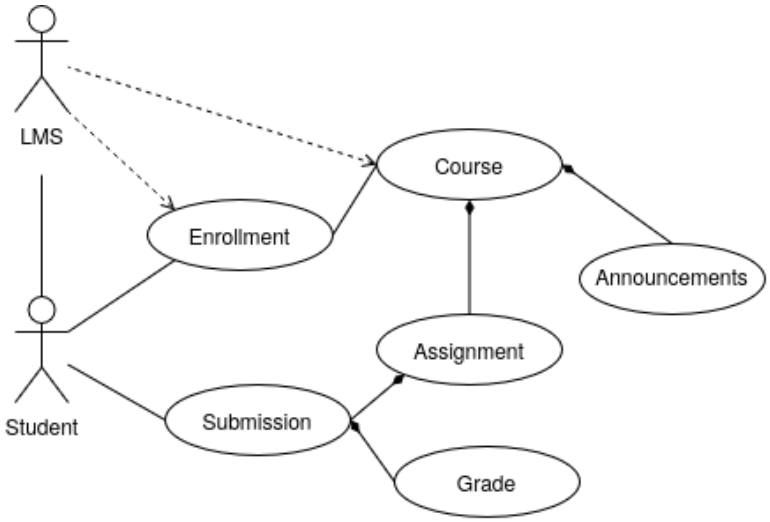
4.3 Self-Review Log

The following review records checks made against the P1 use cases, the expanded scenarios above, and the Lesson 13 criteria in the P2 template.

Use case	Check from Lesson 13	Defect found	How it was fixed
UC-01	Step (0) must describe the initial observable system state.	The P1 scenario began directly with selecting a course.	Added step (0), presenting the student's available courses before an action occurs.
UC-01	Alternate flows must account for normal and exceptional conditions.	The P1 scenario addressed missing information and retrieval failure but did not explicitly cover a selected course being inaccessible.	Added an access-denied alternate flow with a return to course selection.
UC-01	The final response must be observable and satisfy the postcondition.	Supporting-source behavior needed a clearer ending.	Added an observable response identifying the supporting material and confirming its location.
UC-02	Each interaction must identify the student's action and system response.	The P1 flow mixed student actions with internal sorting activities.	Rewritten as paired actor/system interactions focused on what the student requests and sees.
UC-02	A condition that may be false during normal operation belongs in an alternate flow.	Assignments may have no due date, and some students may have no assignments.	Preserved explicit alternate flows for missing due dates and empty assignment lists.
UC-02	A use case must end with a perceivable result.	The P1 scenario ended after displaying an organized list without a detailed follow-up response.	Added an assignment-detail response that confirms the requested details were provided.
UC-04	The use case must have a defined start, end, and postconditions.	UC-04 was not expanded in the provided P1 document.	Defined the trigger, end condition, preconditions, and success/failure postconditions.
UC-04	Exception flows must be described.	No detailed exception flows were available in the P1 document.	Added insufficient-material, retrieval-failure, unauthorized-course, and graded-work-request alternatives.
All three	Analysis steps must avoid implementation details.	References to internal retrieval and processing appeared in the P1 scenarios.	Rephrased the interactions in terms of student requests and observable responses.
All three	Every analyzed use case must trace to functional requirements.	Inconsistent UC-04/UC-05 identifiers appeared in the P1 traceability matrix.	Used the consolidated Section 3.4 numbering and included functional requirement references for each expanded use case.

5 Analysis Models

5.1 Domain model



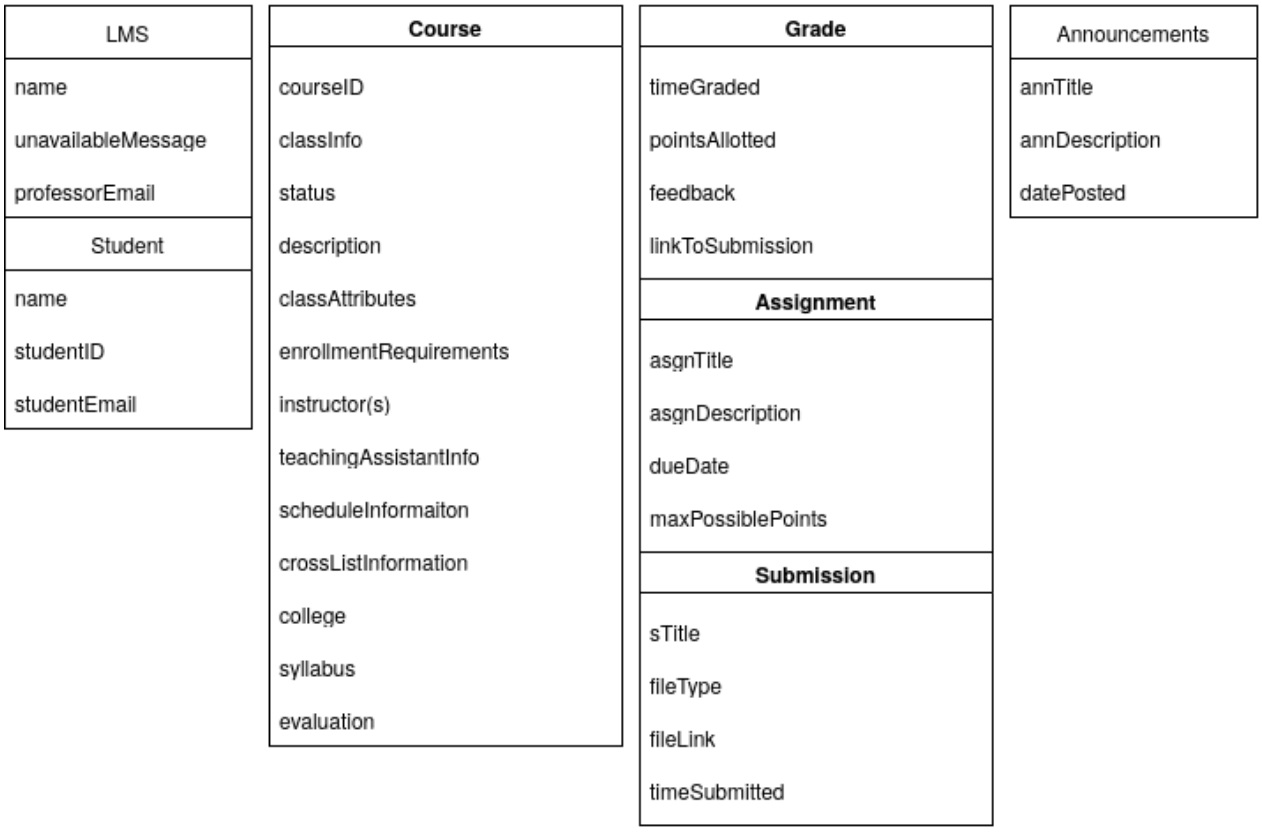


Figure 5.1 Domain model (conceptual classes, attributes, associations). Diagram by Omor Igiehon; source `docs/p2/diagrams/CE 3354 P2 5.1 – Omor Igiehon.drawio`*. The classes marked "added in 5.3" below*

were found missing by cross-check K1 and are recorded here; the figure will be redrawn to include them in P3.

Class	What it represents (one line)	Key attributes	Associations (with multiplicity)
Student	A person enrolled in one or more courses who uses the assistant	name, student ID, email	Student 0..* to 1..* Course (is enrolled in); Student 1 to 0..* Submission (makes); Student 1 to 0..* Question (asks)
Instructor	A person who teaches a course and posts its material	name, instructor ID, email	Instructor 1..* to 0..* Course (teaches)
Course	A course offering the student can see	course ID, title, description, schedule, syllabus, status	Course 1 to 0..* Assignment (has); Course 1 to 0..* Announcement (has); Course 1 to 0..* CourseMaterial (has)
Assignment	A piece of graded work set in a course	title, description (instructions), maximum points, due date (may be empty)	Assignment 1 to 0..* Submission (receives)
Submission	One student's attempt at one assignment	title, file type, time submitted, status	Submission 0..1 to 0..1 Grade (is given)
Grade	The mark given to a submission	score, maximum, date graded	Grade 0..1 to 1 Submission
Announcement	A notice an instructor posts to a course	title, message, date posted	Announcement 0..* to 1 Course
CourseMaterial (added in 5.3)	A page, note or syllabus a student can read in a course	title, content	CourseMaterial 0..* to 1 Course; CourseMaterial 0..* to 0..* Answer (supports)
Question (added in 5.3)	A question a student asks about a course	text, time asked	Question 0..* to 1 Course (is about); Question 1 to 0..1 Answer (receives)
Answer (added in 5.3)	The response to a question, grounded in course material	text, source reference	Answer 0..* to 1..* CourseMaterial (is based on)
PracticeQuestion (added in 5.3)	A self-study question generated from course material (UC-04)	text, topic	PracticeQuestion 0..* to 1 Course (is drawn from)

5.2 Object interaction models

5.2.1 UC-01 Ask Course-Related Questions

(a) Non-trivial steps.

Step (from 4.2)	Non-trivial?	Why
(0) The system presents the student's available courses	No	The same enrollment lookup appears as message 2.2
(1) The student selects a course	No	The student's input; the interface only receives it
(2) The system confirms the course and invites a question	Yes	Needs a check that the student is enrolled in the chosen course (A3 when not)
(3) The student submits a question	No	The student's input; the interface only receives it
(4) The system presents an answer supported by course information, with its source	Yes	Needs the course's material, an answer formed only from it, and the source it came from (A1 when the material is insufficient)
(5) The student asks to examine the supporting material	No	The student's input; the interface only receives it
(6) The system identifies the source and confirms it was located	Yes	Needs the source recorded with the answer

Two interaction objects take part besides the domain classes from 5.1 (Student, Course, CourseMaterial, Question, Answer): **AssistantInterface** (boundary, the same one as in 5.2.2) takes the student's requests and shows responses; **QuestionAnswerer** (control) checks access, gathers course material and forms the answer. It is in control of this interaction.

(b) Scenario table.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	Student	selects a course	the chosen course	AssistantInterface
2.1	AssistantInterface	passes on the chosen course	Course	QuestionAnswerer
2.2	QuestionAnswerer	checks the student is enrolled in the course	Course	Student
2.3	QuestionAnswerer	confirms the selected course	Course	AssistantInterface
2.4	AssistantInterface	shows the course and invites a question	Course	Student
3	Student	submits a question about the course	question text	AssistantInterface
4.1	AssistantInterface	passes on the question	question text, Course	QuestionAnswerer
4.2	QuestionAnswerer	records the question for the course	question text, Course	Question
4.3	QuestionAnswerer	asks for material relevant to the question	Question	Course

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
4.4	Course	looks up its matching material	Question	CourseMaterial
4.5	QuestionAnswerer	forms an answer from the material, noting its source	CourseMaterial	Answer
4.6	QuestionAnswerer	hands over the answer and its source	Answer	AssistantInterface
4.7	AssistantInterface	shows the answer with its source reference	Answer	Student
5	Student	asks to examine the supporting material	Answer	AssistantInterface
6.1	AssistantInterface	passes on the request	Answer	QuestionAnswerer
6.2	QuestionAnswerer	asks for the source the answer is based on		Answer
6.3	QuestionAnswerer	hands over the located material	CourseMaterial	AssistantInterface
6.4	AssistantInterface	shows the source and confirms it was located	CourseMaterial	Student
A1.1	QuestionAnswerer	reports that the material is insufficient	Question	AssistantInterface
A1.2	AssistantInterface	explains that no answer is supported		Student

(c) Analysis sequence diagram.

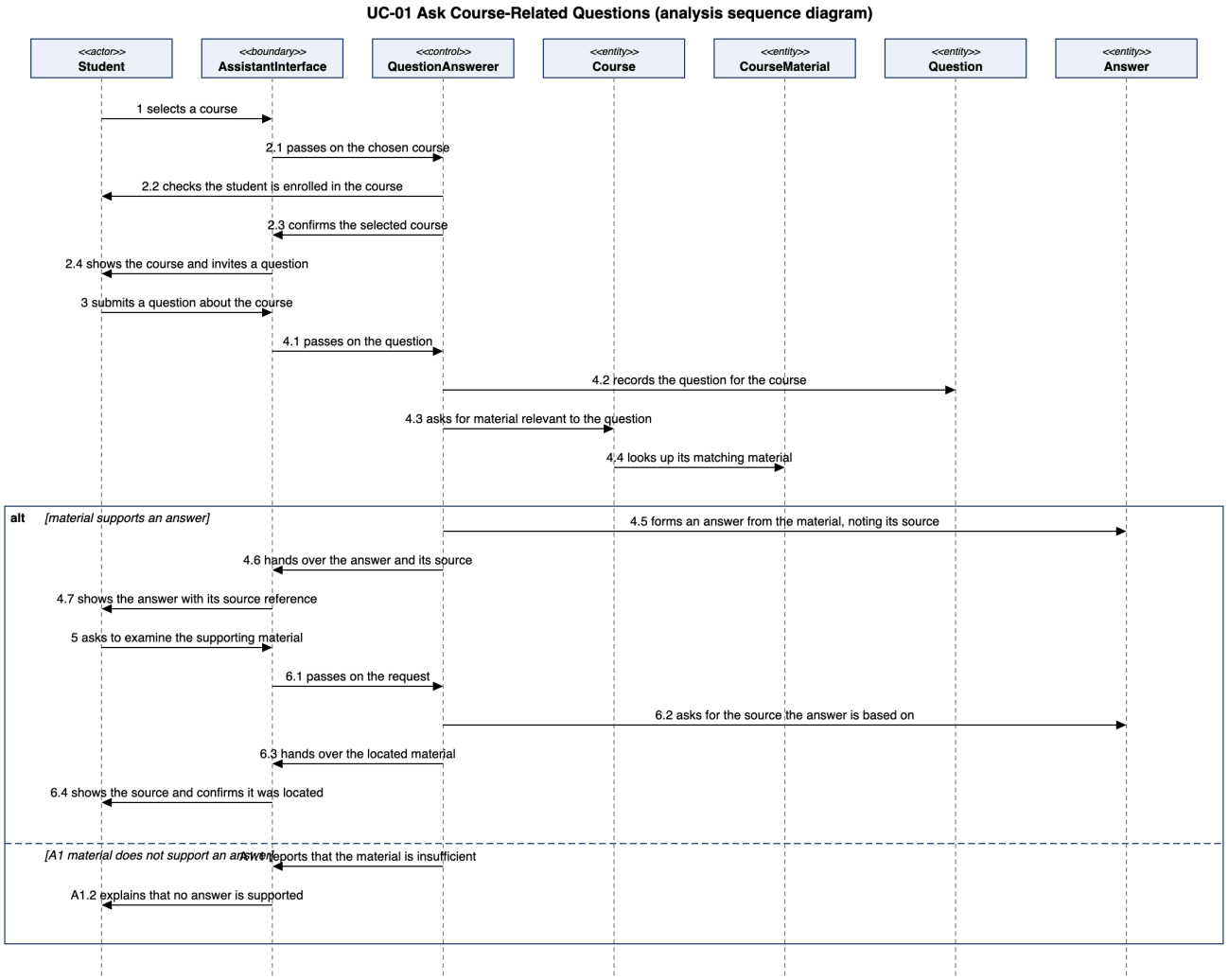


Figure 5.2.1 Analysis sequence diagram for UC-01; message numbers match the scenario table. Source: `docs/p2/diagrams/uc01-analysis-sequence.puml`.

5.2.2 UC-02 Organize Assignments

(a) Non-trivial steps.

Step (from 4.2)	Non-trivial?	Why
(0) The system presents the student's available courses	No	The same lookup of the student's courses appears as message 2.2, so it adds no new interaction
(1) The student requests an organized list of assignments	No	The student's input; the interface only receives it
(2) The system presents the assignments in ascending order of due date, with course names, due dates and submission statuses	Yes	Needs the student's enrolled courses, each course's assignments and the student's submission for each one, then an ordering that puts undated assignments last (A2)

Step (from 4.2)	Non-trivial?	Why
(3) The student requests more information about one assignment	No	The student's input; the interface only receives it
(4) The system presents the assignment's instructions, due date and submission status	Yes	Needs the chosen assignment's details and the student's submission for it
A3 at step (2): assignment information cannot be retrieved	Yes	A course's records fail to arrive, and the student must be told without a partial list shown as complete

Two interaction objects take part besides the domain classes from 5.1 (Student, Course, Assignment, Submission):

- **AssistantInterface** (boundary): takes the student's requests and shows the system's responses.
- **AssignmentOrganizer** (control): gathers the student's assignments across courses, puts them in order and fetches one assignment's details. It is in control of this interaction.

(b) Scenario table. Rows 2.3 to 2.5 repeat for each enrolled course and each of its assignments.

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
1	Student	asks for an organized list of assignments from their enrolled courses	the request	AssistantInterface
2.1	AssistantInterface	passes on the request to organize the student's assignments	Student	AssignmentOrganizer
2.2	AssignmentOrganizer	asks for the courses the student is enrolled in		Student
2.3	AssignmentOrganizer	asks the course for its published assignments		Course
2.4	AssignmentOrganizer	asks the assignment for the student's submission status	Student	Assignment
2.5	Assignment	looks up the student's submission	Student	Submission
2.6	AssignmentOrganizer	orders the assignments by due date, placing undated ones last and marking them as having no deadline	due dates	AssignmentOrganizer
2.7	AssignmentOrganizer	hands over the organized list	each assignment's course name, due date and submission status	AssistantInterface
2.8	AssistantInterface	shows the organized list	the organized list	Student

Step	Subject (who acts)	Subject action (what it does)	Other data / objects (what is used)	Object acted upon (who receives)
3	Student	asks for more information about one assignment	the chosen assignment	AssistantInterface
4.1	AssistantInterface	passes on the request for that assignment's details	the chosen assignment	AssignmentOrganizer
4.2	AssignmentOrganizer	asks the assignment for its instructions, due date and the student's submission status	Student	Assignment
4.3	Assignment	looks up the student's submission	Student	Submission
4.4	AssignmentOrganizer	hands over the assignment's details	instructions, due date, submission status	AssistantInterface
4.5	AssistantInterface	shows the assignment's details and confirms they have been provided	the assignment's details	Student
A3.1	Course	reports that its assignments cannot be retrieved		AssignmentOrganizer
A3.2	AssignmentOrganizer	reports that the assignment list could not be completed	the course's name	AssistantInterface
A3.3	AssistantInterface	explains that assignment information is temporarily unavailable, without showing a partial list		Student

(c) Analysis sequence diagram.

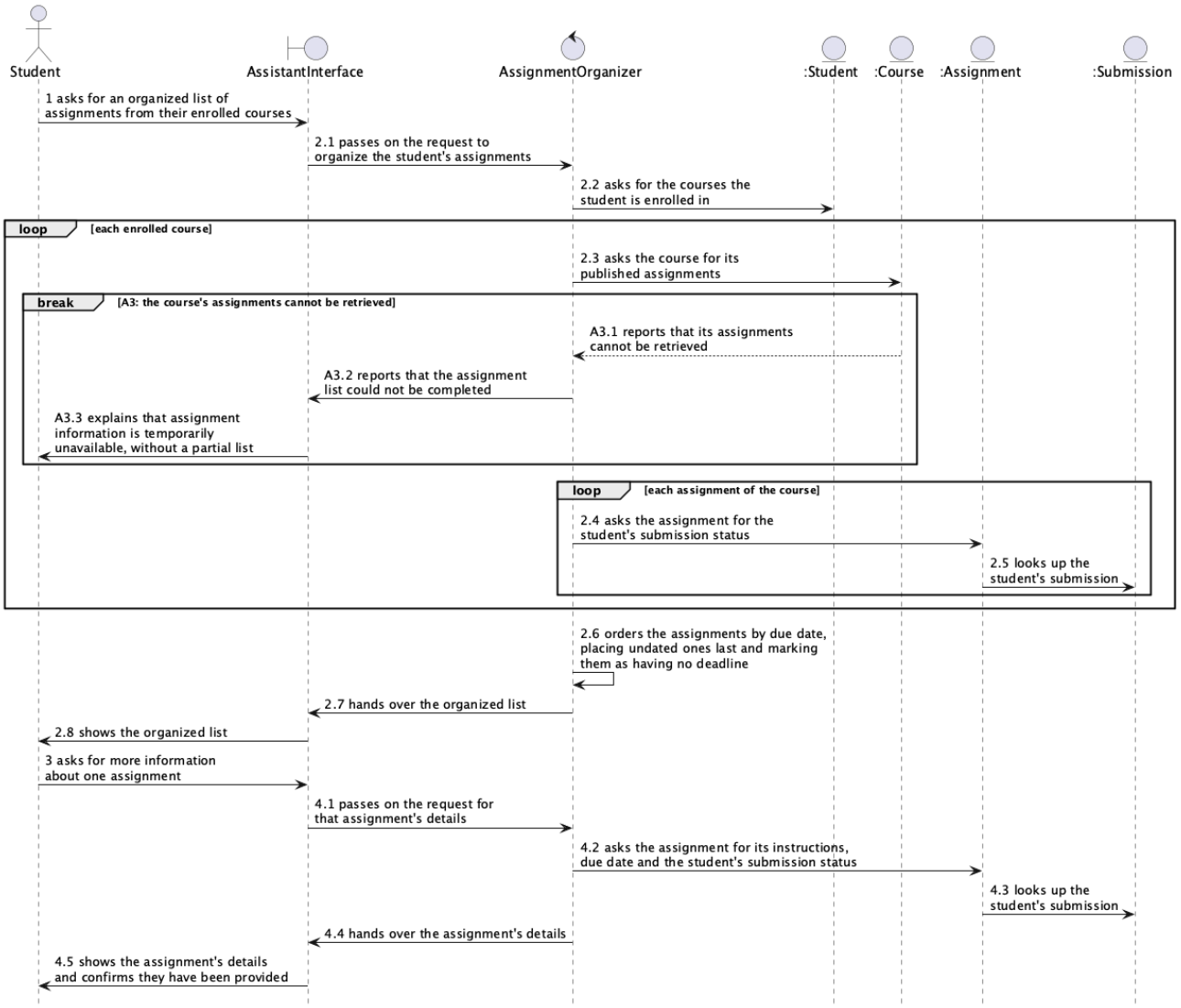


Figure 5.2.2 Analysis sequence diagram for UC-02; message numbers match the scenario table. Source: `docs/p2/diagrams/uc02-analysis-sequence.puml`.

The last messages make the UC-02 postconditions true: 2.8 shows the available assignments in order of due date, 4.5 shows one assignment's details and confirms them, and on alternate flow A3, A3.3 gives the student the explanation.

5.3 Consistency cross-check

Check	Result (pass, or what failed)	Fixed in
K1 Every item in an actor column (4.2) is an attribute in the domain model (5.1)	Failed at first: the domain model drawn for 5.1 had no class for the question, the answer with its source, the course pages a student reads, or practice questions, although UC-01 and UC-04 name them	5.1: Question, Answer, CourseMaterial and PracticeQuestion added to the class table

Check	Result (pass, or what failed)	Fixed in
K2 Every object on a sequence diagram is a domain class or a declared interaction object	Pass: UC-01 uses Student, Course, CourseMaterial, Question, Answer plus AssistantInterface and QuestionAnswerer; UC-02 uses Student, Course, Assignment, Submission plus AssistantInterface and AssignmentOrganizer, all declared in 5.2	n/a
K3 Every message on a sequence diagram corresponds to exactly one scenario-table row	Pass for UC-01 (20 rows, 20 messages) and UC-02 (checked in the review of PR #3)	n/a
K4 Every non-trivial step in 5.2 (a) has at least one scenario-table row	Pass: UC-01 steps 2, 4, 6 (rows 2.1 to 6.4) and A1; UC-02 steps 2, 4 and A3	n/a
K5 Every analyzed use case traces to at least one functional requirement	Pass: UC-01 to FR-01, FR-02, FR-03, FR-07; UC-02 to FR-02, FR-04, FR-05, FR-07; UC-04 to FR-08	n/a
K6 The last messages of each diagram make the use case postcondition true	Pass: UC-01 ends with 6.4 showing the source (or A1.2 explaining no answer); UC-02 ends with 4.5 confirming the details (or A3.3 explaining the failure)	n/a

6. High-Level Design

6.1 Quality Attribute Scenarios

The following are verifiable design targets, not claims that the prototype has passed testing. The sources of these scenarios are the revised P2 requirements (Section 3) and the P3 development plan.

Quality attribute	Artifact	Response	Response measure	From
QAS-01: Performance (QR-01). Twenty students send ordinary course questions concurrently.	Chat panel, `POST /api/chat`, agent orchestrator, LMS adapter, Claude API	Accept concurrent questions and provide complete responses without blocking unrelated sessions.	**95 of 100 requests finish within 5.0 seconds**, measured from HTTP request start to full HTTP response, in batches of 20 concurrent requests; log p50, p95, error count and test configuration. This is a **target**, pending S-01 spike results.	QR-01; UC-01; spike S-01

Quality attribute	Artifact	Response	Response measure	From
QAS-02: Access control (QR-03). An unauthorized user requests another student's grades, or calls the endpoint without valid identity.	Chat API identity boundary, MoodleLMS adapter and Moodle access controls	Deny requests without verified identity or permission and disclose no protected records.	**20/20** negative authorization tests disclose **zero** protected records; reject invalid/missing credentials with HTTP 401/403 or equivalent. **Not met in the current prototype:** the API has no per-student authentication and uses one configured test token.	QR-03; FR-02; UC-01; ADR-05
QAS-03: Failure handling (FR-07). Moodle is unavailable or its REST API fails while a student asks about assignments.	LMS adapter, agent orchestrator, FastAPI and chat panel	Show a clear failure notice without fabricating assignment data or leaking credentials.	In **at least 19/20** simulated outages, show a non-sensitive error **within 10 seconds**; **0/20** label invented data as retrieved from Moodle. This is a proposed P3 test, not a measured result.	FR-07; UC-02; P3 backlog

Constraints shaping the architecture: CON-01 requires integration with the team's self-hosted Moodle 5.3 LMS, so academic data is retrieved through Moodle REST rather than Canvas. **CON-02** limits development to the course team's available resources, favoring a lightweight layered prototype and avoiding separately operated microservices. The existing Moodle-token design and external Claude dependency also require the security measures described in Section 6.4.

6.2 Architectural Style and Alternatives

Selected architecture: layered client-server with an external AI service (ADR-01). The presentation layer is the Moodle-embedded chat UI (`web/embed.js` and `web/chat.html`). The API/application layer is FastAPI (`server.py`) and the agent orchestrator (`agent.py`). The integration layer is the seven read-only tools plus `lms.py`. Moodle 5.3 and PostgreSQL maintain authoritative academic records; Anthropic Claude performs external AI inference. These are logical layers, not necessarily separate physical hosts.

Style	Drivers it serves	What it costs	Verdict
Layered client-server + external AI	UC-01 through UC-04, QAS-01, separation of presentation/orchestration/LMS access, `MockLMS` testing, Moodle integration	Network round trips, latency and paid inference; external transfer of selected student context; single service failure point	**Selected**: simple fit for the actual Python/FastAPI prototype and ADR-01
Monolithic Moodle plugin	Fewer independent endpoints; tight LMS UI integration	Couples model changes, secrets and deployment to Moodle plugin internals; harder independent offline testing	**Rejected**: unnecessary Moodle coupling

Style	Drivers it serves	What it costs	Verdict
Event-driven microservices with queues	Can isolate components at larger scale and absorb asynchronous workloads	Multiple deployments, queue state, ordering, tracing and debugging for a small request/response prototype	**Rejected**: excessive complexity under CON-02

The chosen style allows the chat interface to change without rewriting Moodle access, isolates the real/mock LMS behind a consistent tool interface, and keeps LMS records in Moodle. It does **not** by itself meet the latency or authentication goals. In P3 the team plans per-student LTI 1.3 identity, source references, cross-course deadline logic and performance improvements.

6.3 Component View

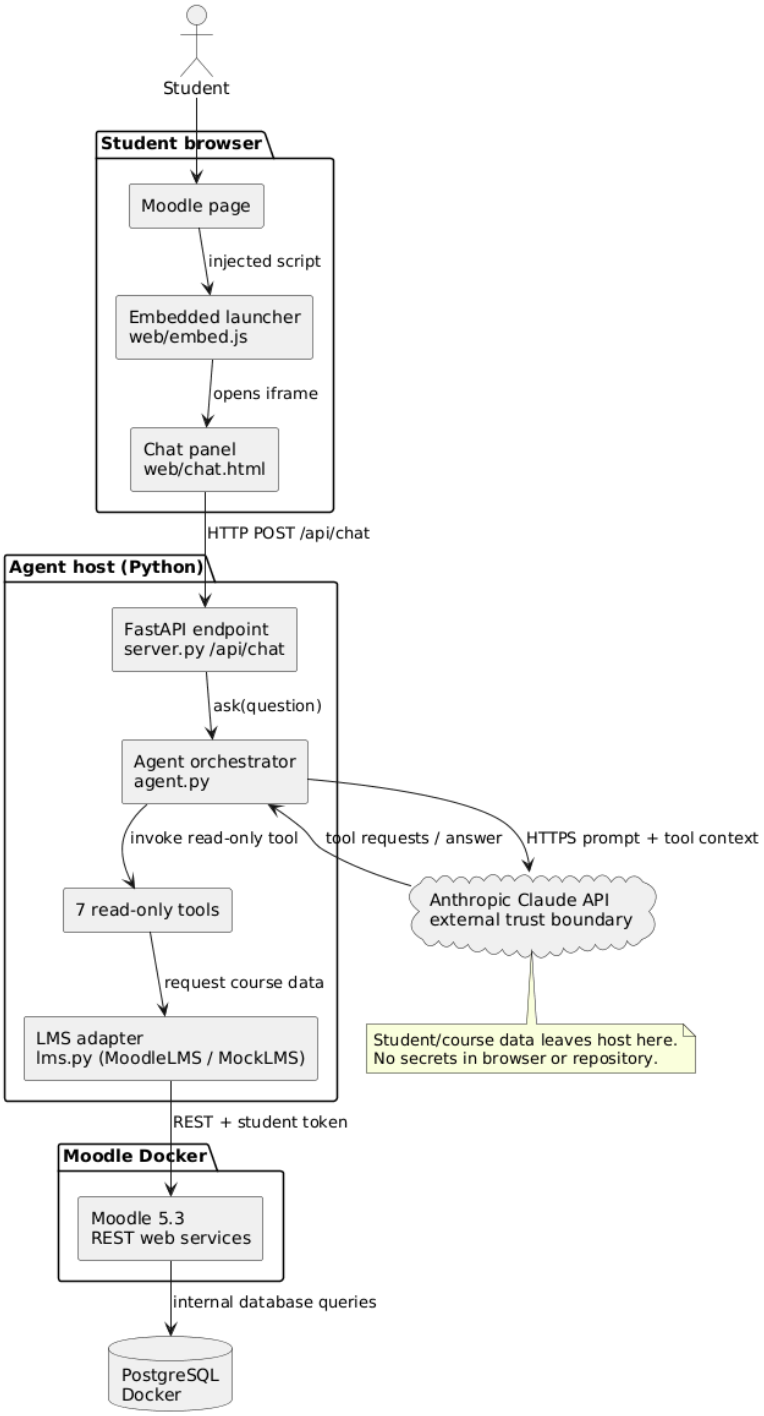


Figure 6.3. UML component view. Editable source: `docs/p2/diagrams/component-view.puml`.

Component	Responsibility	Provides	Requires	Answers to (UC / QAS / ADR)
Moodle embedded launcher (`web/embed.js`)	Make the assistant available inside Moodle pages	Button and embedded chat iframe	Moodle page, reachable chat host	UC-01, QR-02, ADR-04

Component	Responsibility	Provides	Requires	Answers to (UC / QAS / ADR)
Chat panel (`web/chat.html`)	Capture requests and display answers or errors	Question input, answer display and status	FastAPI chat endpoint	UC-01–UC-04, QAS-03, ADR-04
FastAPI (`server.py`)	Validate requests, manage temporary sessions, serve frontend and API	`POST /api/chat`, `/api/reset`, `/api/health`	Agent orchestrator, in-memory session state	UC-01–UC-04, QAS-01, QAS-02, ADR-01
Agent orchestrator (`agent.py`)	Run Claude tool-calling loop and tutoring instructions	AI answer generation and tool selection	Claude API, seven LMS tools	UC-01–UC-04, QAS-01, ADR-01, ADR-03
Seven read-only LMS tools (`agent.py`)	Expose course, assignment, page and announcement lookups	Typed retrieval operations	LMS adapter methods	UC-01–UC-03, ADR-03
LMS adapter (`lms.py`)	Convert retrieval operations to Moodle REST or mock results	`MoodleLMS` / `MockLMS` interface	Moodle REST and configured test student token	UC-01–UC-03, QAS-02, ADR-02, ADR-05
Moodle 5.3 REST service (external LMS subsystem)	Enforce LMS-side permissions and supply student-visible records	Moodle course-data REST API	PostgreSQL LMS database	UC-01–UC-03, ADR-02, ADR-05
Anthropic Claude API (external provider)	Generate responses and tool requests	Model inference	Selected prompts, question/context	UC-01, UC-04, QAS-01, ADR-03

Interaction flow: student opens the embedded chat; the chat posts to `/api/chat`; FastAPI calls the agent; the agent sends a prompt/tool schema to Claude; the model requests a read-only tool when needed; the adapter fetches accessible Moodle data; the resulting tool content is passed back to Claude for an answer; FastAPI returns that answer to the panel. A question may cause multiple model and Moodle calls. The browser does not directly query PostgreSQL or call Claude.

Known implementation gaps: source citations, dedicated seven-day cross-course aggregation, measured response-time compliance, and per-student authentication are not yet implemented or verified.

6.4 Deployment View and Trust Boundaries

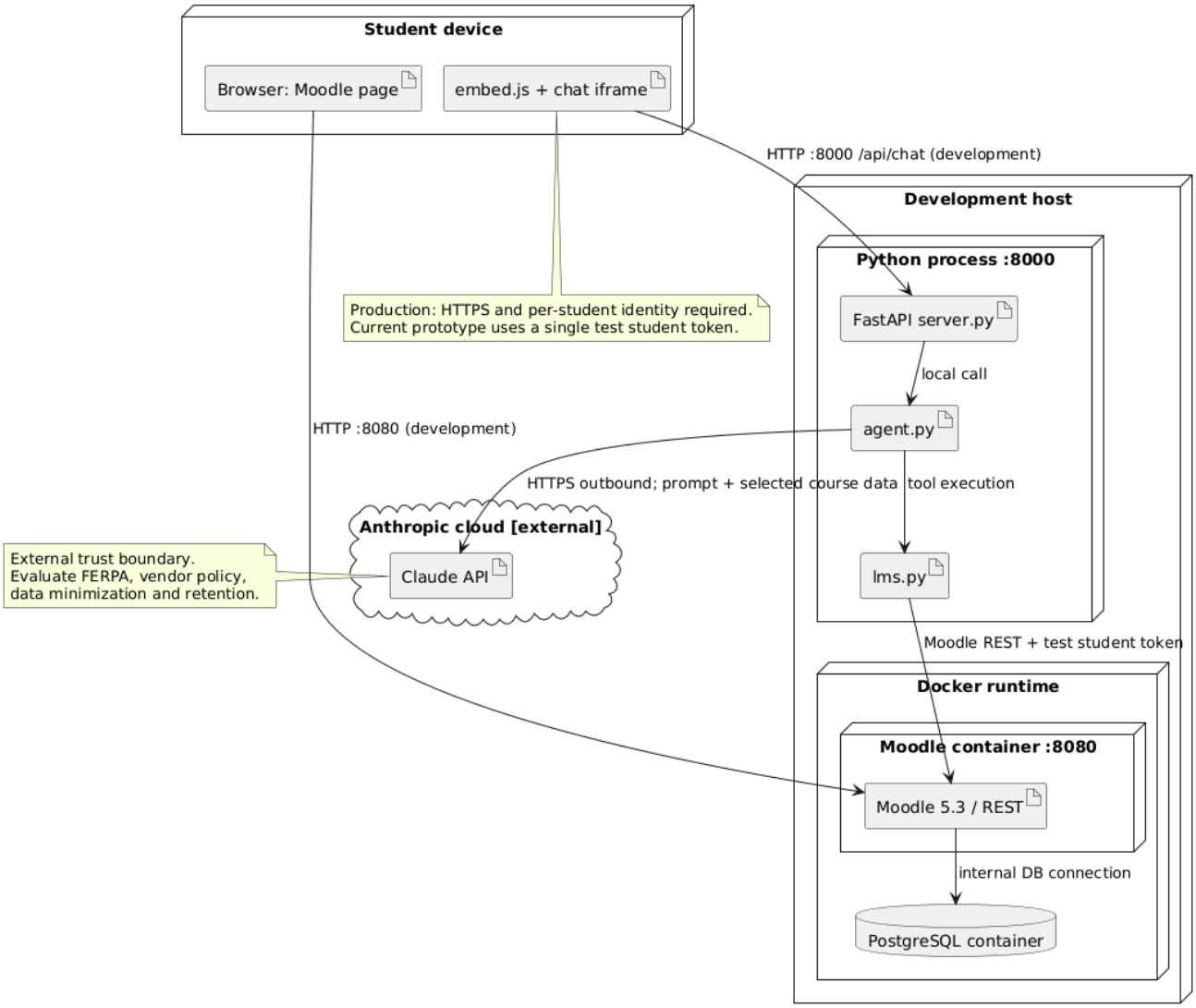


Figure 6.4. UML deployment view. Editable source: `docs/p2/diagrams/deployment-view.puml`.

Node / boundary	Deployed elements	Communication and data
Student device/browser	Moodle web page, injected `embed.js`, iframe chat page	Browser loads Moodle over local HTTP port 8080 and chat/API over local HTTP port 8000 in the prototype. Production must use HTTPS and appropriate origin controls.
Local development host	Python FastAPI `server.py`, `agent.py`, `lms.py`; in-memory `sessions` and per-session locks	`/api/chat` invokes orchestration; LMS adapter calls Moodle REST. The server retains chat messages in RAM while running (up to 200 sessions, with eviction); it does **not** persist a conversation database. Current configuration uses one test student's token, **not** real per-user authentication.
Docker: Moodle container	Moodle 5.3, REST web services	Authoritative LMS authorization and course operations.

Node / boundary	Deployed elements	Communication and data
Docker: PostgreSQL container	Moodle database	Stores course, user, assignment, grade, page and announcement data; only Moodle should query it.
External Anthropic infrastructure	Claude model/API	Agent sends student prompts and selected course/tool context over HTTPS. **Data leaves the team's host and control at this boundary.** API credentials must remain server-side.

Security/privacy implications: Sending identifiable course or grade information to Anthropic may involve education records; the team must evaluate institutional policy, FERPA applicability, vendor terms, retention, consent and minimization before using real student data. The chat API currently has **no per-student authentication check**, and a browser-supplied session ID refers to server-side chat history, not a verified student identity. All chat requests use the same configured Moodle token. Therefore the prototype is suitable for synthetic/test data, not proof of production FERPA compliance. LTI 1.3 student identity and authorization checks are planned for P3. No passwords, tokens or API keys belong in diagrams, repository commits or client JavaScript.

6.5 Analysis-to-Component Mapping

The table maps analysis objects (Section 5) to **components of this design**, not merely to the external record owner. Omor's UC-01 object names and final Section 5.1 model should be checked for exact spelling before the team assembles its PDF. "Split" means the analysis responsibility is implemented by more than one component; "external" means records are hosted by Moodle, not reimplemented in our agent.

Analysis object / concept	Design component(s) from 6.3	Allocation	Rationale
`AssistantInterface` (UC-02)	Moodle embedded launcher + chat panel (`web/embed.js`, `web/chat.html`)	Split	Opens the assistant, accepts student input and displays organized assignments.
`AssignmentOrganizer` (UC-02)	Agent orchestrator (`agent.py`) + read-only tools + LMS adapter (`lms.py`)	Split	Coordinates course/assignment requests; underlying data retrieval remains in the adapter. Chronological ordering and dedicated seven-day aggregation are not fully implemented.
Student	LMS adapter (`lms.py`) using Moodle user identity; FastAPI for temporary chat session	External record / split	Moodle owns student records; current prototype has only one configured test student token, not verified chat identity.
Course / Enrollment	LMS adapter (`lms.py`) via Moodle course retrieval	External record	Course/enrollment data resides in Moodle, exposed by the adapter.

Analysis object / concept	Design component(s) from 6.3	Allocation	Rationale
Assignment / Submission	LMS adapter (`lms.py`) and read-only assignment tools	External record	Moodle supplies deadlines, statuses and assignment instructions.
Grade	LMS adapter (`lms.py`) and course/assignment tools	External record	Grades remain authoritative in the Moodle gradebook.
Announcement	LMS adapter (`lms.py`) and announcement tool	External record	Moodle forum/announcement records are accessed read-only.
Course module / Page / Syllabus	LMS adapter (`lms.py`) and module/page/syllabus tools	External record	The adapter normalizes course content for use in answers.
Student question / Chat session	Chat panel + FastAPI in-memory session dictionary	Split	Browser submits question; server retains temporary history, not a persistent database.
Generated answer / tool request	Agent orchestrator + external Claude API	Split / external service	Claude generates responses/tool requests; the agent validates and executes defined LMS tools.

6.6 Design Principles

Principle	Design evidence	Benefit / trade-off
Separation of concerns	`web/chat.html` and `web/embed.js` own user interaction; `server.py` owns HTTP endpoints/session handling; `agent.py` owns model workflow; `lms.py` owns Moodle access.	Changes remain localized; additional layer boundaries require clear interfaces.
Information hiding	Moodle tokens and REST request parameters are encapsulated in `MoodleLMS` ; the chat UI calls `/api/chat` rather than handling Moodle credentials.	The client cannot directly read tokens; backend must still authenticate users before real deployment.
Low coupling	`agent.py` calls the same seven methods whether `make_lms()` returns `MoodleLMS` or `MockLMS` ; the browser speaks only to the FastAPI contract.	Supports offline tests and possible future LMS adapters; actual adapter remains Moodle-specific.
High cohesion	The agent manages inference/tool choices; the LMS adapter performs data retrieval; chat files manage presentation; Moodle maintains its academic database.	Each module has a focused responsibility, making failures easier to locate.
Design for change	`MockLMS` and `MoodleLMS` share a tool interface; P3 additions (LTI identity, citations, 7-day aggregation) can be introduced without replacing the UI and database.	Identity and citation contracts still need explicit implementation and testing.
KISS (Keep It Simple)	One FastAPI service, an embedded browser panel, seven read-only retrieval tools, Moodle containers and one external model; rejected queued microservices and a custom Moodle plugin.	Less operational complexity, but sequential external calls may affect QAS-01 latency.

7 Design Rationale

7.1 Architecture decision records

Each record names the drivers that forced the decision, the options the team actually weighed, what was chosen and what that choice costs. ADR-01 records the architectural style chosen in section 6.2.

ADR-01 Layered client-server architecture with an external AI service

Field	Record
Status	Accepted
Date and owner	Decided Nov 8, 2026 by Ahmad Zghier (section 6.2, commit `9aec1ea`); recorded by Daniel Habib
Context (drivers)	At least 95 of 100 requests must finish within 5.0 seconds with 20 concurrent requests (QAS-01), 100% of 20 negative authorization tests must return no protected information (QAS-02), and at least 19 of 20 injected LMS failures must produce a visible, non-sensitive error within 10 seconds (QAS-03). The system must work with Moodle (CON-01) within the team's resources and one term (CON-02).
Options considered	(a) Layered client-server: the chat panel and launcher in the student's browser, an agent server (chat endpoint, agent orchestrator, read-only tools), an LMS adapter in front of Moodle, and Claude as an external service. (b) Monolithic Moodle plugin: the whole AI workflow built into Moodle. (c) Event-driven microservices: separate services and queues for chat, retrieval, tool execution and inference.
Decision	(a) Layered client-server with an external AI service.
Consequences	Favors modifiability and testability: the chat interface, agent and LMS adapter change independently, `MockLMS` stands in for Moodle in offline tests, Moodle stays the single source of truth, and student data leaves for Claude along one explicit path (QAS-02). An LMS failure surfaces in one layer, the adapter, where it can become a clear message (QAS-03). Cost: every question passes through each layer synchronously, plus one or more round trips to Claude, so delays add up (QAS-01, measured by S-01). The agent server is a single process that keeps chat sessions in memory, so it cannot scale out and loses chat history on restart. Compared with (b), it is one more service to run and secure beside Moodle; compared with (c), it gives up isolation at large scale.
Evidence	The prototype already follows these layers: `web/` (presentation), `server.py` and `agent.py` (application), `lms.py` (LMS integration), commits `b5c18f7` and `347886c`. S-01 (section 8) measures the latency cost against QAS-01.
Related	ADR-03, ADR-04, QAS-01, QAS-02, QAS-03, S-01

ADR-02 Self-hosted Moodle as the LMS

Field	Record
Status	Accepted
Date and owner	Decided Nov 7, 2026 by John Bramlet (prototype commit `b5c18f7`); recorded by Daniel Habib
Context (drivers)	The agent must read a student's courses, assignments and submission status through an LMS API (FR-02, FR-04) with that student's own access (QAS-02), and the team must be able to run it within the course's resources (CON-02). P1 planned on Canvas but left the final platform open (Q1).
Options considered	(a) Canvas, free version: does not allow third-party services such as our agent to connect, so the agent could not read any course data. (b) Google Classroom: needs a Google Cloud OAuth app before any API call. (c) Self-hosted Moodle 5.3 with PostgreSQL in Docker, read through Moodle's web-service API with the student's own token.
Decision	(c) Self-hosted Moodle 5.3. CON-01 now names Moodle, which answers Q1.
Consequences	Favors feasibility and schedule: the API works today, costs nothing, and any member can run Moodle on a laptop. Cost: it is not the LMS UT Dallas students use, so the agent cannot reach their real courses and all course content is our own sample data (R3). The team hosts, seeds and upgrades Moodle itself. The LMS adapter (`lms.py`) is Moodle-specific, so supporting Canvas later means writing a second adapter behind the same seven tool methods.
Evidence	Working prototype against Moodle: commits `b5c18f7` (agent and Moodle adapter), `f019be3` (Moodle install fix), `c5ee6e5` (sample course seed). Canvas free-version block: `ONBOARDING.md`, section 1.
Related	ADR-04, ADR-05, CH-02, CON-01, Q1, R1, R3

ADR-03 Fetch course data on demand through read-only tools

Field	Record
Status	Accepted
Date and owner	Decided Nov 7, 2026 by John Bramlet (prototype commit `b5c18f7`); recorded by Daniel Habib
Context (drivers)	Answers must come only from the student's own enrolled courses (FR-02, FR-03) and reflect live assignment status (FR-04 to FR-06), at least 95 of 100 requests must finish within 5.0 seconds with 20 concurrent requests (QAS-01), and accuracy must reach 95% on the evaluation set (QR-04). LLM cost is a known risk (R4).
Options considered	(a) Tool calling on demand: the model chooses among seven read-only tools (courses, assignments, assignment instructions, syllabus, modules, pages, announcements) and the agent reads Moodle live for each question. (b) Preload: fetch all of the student's course content and send it with every question. (c) Search index (RAG): copy course content into an index ahead of time and send the best-matching passages with each question.
Decision	(a) Tool calling on demand. Every tool only reads; nothing writes to the LMS.

Field	Record
Consequences	Favors freshness and least privilege: due dates, submission status and grades are read at the moment of asking, each read uses the student's own access, and no copy of student records is stored on our side. Only what a question needs is sent to the model provider, where preloading would send the whole course each time. Cost: one question can take several model turns, each waiting on the model and on Moodle (listing assignments makes one Moodle call per assignment), which puts QAS-01 at risk and multiplies API cost per question (R4). Answer quality depends on the model picking the right tools, and with no full-text search a fact on an unvisited page can be missed (R2).
Evidence	Agent loop in `agent.py` and per-assignment status calls in `lms.py` (commit `b5c18f7`). Spike S-01 (section 8) measures this design's response time against QAS-01.
Related	ADR-01, S-01, QAS-01, QR-04, FR-03, R2, R4, R6, R8

ADR-04 Reach students through a chat bubble inside Moodle

Field	Record
Status	Accepted
Date and owner	Decided Nov 7, 2026 by John Bramlet (commit `347886c`); recorded by Daniel Habib
Context (drivers)	Students already know their LMS but cannot be assumed to know our agent (section 2.3), and they need a comparable experience across devices (section 2.4). First-time students must find their assignments quickly (QR-02).
Options considered	(a) Terminal app: kept for testing, unusable for most students. (b) Standalone web page: a separate site students would have to find and open. (c) Chat bubble: a script added once through Moodle's "Additional HTML" setting puts a button on every signed-in Moodle page that opens our chat page in a panel. (d) Moodle plugin written in PHP and installed into Moodle.
Decision	(c) Chat bubble inside Moodle. The standalone page stays available at the agent server's root as a fallback.
Consequences	Favors usability: the assistant sits where students already are, with no new site to find, and the panel fits phone-width screens. The agent stays a separate service that can change without reinstalling anything in Moodle, which a plugin (d) would require. Cost: a Moodle administrator must allow our script on every page, which a university's LMS team would have to approve. The panel is served from our server, not Moodle, so it does not inherit the student's Moodle sign-in (see ADR-05) and does not know which course page the student is on. Our server must be reachable from every student's browser.
Evidence	Chat page and bubble: commit `347886c` (`web/chat.html`, `web/embed.js`, `server.py`); setup in `README.md`, section 4.
Related	ADR-02, ADR-05, QR-02, R7

ADR-05 Read the LMS only with the student's own credential

Field	Record
Status	Accepted: a configured test token for the prototype, LTI 1.3 sign-in for P3
Date and owner	Decided Nov 7, 2026 by John Bramlet (commit `b5c18f7`); recorded by Daniel Habib
Context (drivers)	All 20 negative authorization tests must return no protected information (QAS-02), the agent may read only the student's enrolled courses (FR-02), student records fall under FERPA, and OIT needs the agent kept inside the access a student already has (section 2.2, Q3). P1 also named campus sign-in as a constraint (section 2.4).
Options considered	(a) One test student's token, set in the server's configuration. (b) Each student pastes their own Moodle security key into the chat. (c) LTI 1.3: Moodle launches the agent and tells it which student is signed in.
Decision	Every LMS read uses the signed-in student's own Moodle token, so Moodle itself decides what the agent may see. (a) for the P2 prototype only, (c) as the P3 sign-in. (b) was rejected because it asks every student to find and handle a credential.
Consequences	Favors security and privacy: the agent never holds more access than the student, so a bug or a manipulated prompt cannot reveal another student's grades, because Moodle refuses the request. Cost: the prototype cannot demonstrate QAS-02 across students, since every chat uses one test student's token and the chat API does not yet check who is calling. LTI 1.3 identifies the student but does not by itself give the agent that student's Moodle token, so P3 must add a way to obtain one (Q6); falling back to one privileged account would move QAS-02 enforcement into the agent and supersede this record. Campus sign-in (section 2.4) is met only through the student's Moodle sign-in.
Evidence	`lms.py` sends every Moodle call with the student's token (commit `b5c18f7`); prototype limits in `README.md`, section 4. Per-student sign-in: none yet.
Related	ADR-02, ADR-04, CH-10, QAS-02, FR-02, Q3, Q6, R7

7.2 Trade-off summary

Quality favored	Architectural move	Cost accepted	Recorded in
Modifiability, testability	Layered client-server with Claude as an external service	Delays add up across layers (QAS-01); one in-memory agent server that cannot scale out	ADR-01
Feasibility, schedule	Self-host Moodle 5.3 instead of Canvas	Not the LMS students use; sample data only; the team hosts it; a Moodle-specific adapter	ADR-02
Data freshness, least privilege	Read course data live through read-only tools the model calls	Several model turns per question: response time (QAS-01, measured by S-01) and API cost (R4)	ADR-03
Usability	Chat bubble on every Moodle page	An admin-approved site-wide script; no inherited sign-in or course context; a server every student must reach	ADR-04

Quality favored	Architectural move	Cost accepted	Recorded in
Security, privacy	Every LMS read uses the student's own token	Per-student sign-in still to build; the prototype runs on one test token	ADR-05

8 Prototype Evidence: Architecture Spike

Field	Content
Spike ID and title	S-01 Response time under 20 concurrent students
Risk or decision it addresses	R6 / ADR-03 / QAS-01
Question (answerable yes or no)	Does the agent answer at least 95 of 100 typical student questions within 5.0 s when 20 students ask at the same time?
Set-up	Agent server (FastAPI, one process) and self-hosted Moodle 5.3 with PostgreSQL in Docker on one laptop (Apple M5 Pro, 48 GB); Claude Opus 5.5 at medium effort; CS101 sample course read with the test student's token; 10 typical questions rotated. Folder: /spikes/S-01-latency/
How it was measured	run_spike.py sent 100 requests to POST /api/chat in 5 batches of 20 concurrent requests and timed each from request start to full response, as QAS-01 specifies
Result (numbers)	100/100 succeeded. p50 8.59 s, p95 13.26 s, max 14.52 s, min 5.21 s. 0 of 100 within 5.0 s. Mean 2.6 LMS lookups per question. A question needing no LMS lookup ("What is a loop counter?") still took a median of 5.95 s, and batch medians stayed between 8.2 and 9.1 s, so model time, not Moodle or concurrency, dominates
What it changed or confirmed	Answer: **No**. Confirms R6 and the response-time cost recorded in ADR-03: QAS-01 is not met by the current design. For P3 the team will either measure time to the first visible words (streaming), route simple questions to lower effort or a faster model, combine LMS lookups, or revise QR-01 with this evidence (Q5)
Where to see it	Branch spikes/s-01-starter, commits baed82d (script) and c275bb4 (results); /spikes/S-01-latency/results.csv, summary.txt, README.md

9 Requirements Traceability

9.1 Extended traceability matrix

This matrix uses the consolidated UC-01 through UC-04 list in Section 3 and the expanded scenarios in Section 4. UC-05 is not retained as a separate goal; practice questions map to UC-04. Sources are

pinned because PR #7 targets `docs/team-onboarding`, which does not yet contain these merged-main sections.

Component names and analysis concepts follow Section 6.3 / 6.5. `AssistantInterface` and `AssignmentOrganizer` are supported by Daniel's UC-02 analysis draft in PR #3. Other analysis names below are concepts from Section 6.5, not invented or approved domain classes. Final Section 5.1 and UC-01 object names were unavailable in the inspected branches and still need reconciliation. ADR references use Daniel's Section 7 draft in PR #2 and remain subject to its review.

“Partial” means relevant code exists, not that the acceptance test passed. “Unverified” means no acceptance evidence was available. Source inspection uses the unchanged `server.py`, `agent.py`, `lms.py` and `web/` in PR #7's original source commit `20f021f`.

Requirement	Use case(s)	Analysis objects / concepts	Design components	QAS / ADR	Coverage and verification gap
FR-01 Natural-language input	UC-01	Student question / Chat session	Chat panel; FastAPI	ADR-01, ADR-04	Partial: UI posts messages and API accepts up to 4,000 characters. Required 10-question receipt test not recorded.
FR-02 Only enrolled student's courses	UC-01–UC-04	Student, Course / Enrollment	LMS adapter; Moodle REST; FastAPI identity boundary	QAS-02; ADR-02, ADR-05	Partial: Moodle calls use one configured token. Chat caller identity is not verified; per-student isolation remains missing.
FR-03 Course-grounded answers	UC-01, UC-04	Course module / Page / Syllabus; Generated answer	Agent orchestrator; read-only tools; LMS adapter; Claude	ADR-03; QR-04	Partial: tools and prompt request grounding. No 8-question acceptance evidence; source references required by UC-01 are missing.

Requirement	Use case(s)	Analysis objects / concepts	Design components	QAS / ADR	Coverage and verification gap
FR-04 Assignment status retrieval	UC-02, UC-03	AssignmentOrganizer; Student, Course, Assignment, Submission	Agent orchestrator; assignment tools; LMS adapter; Moodle REST	ADR-02, ADR-03	Partial: course-specific assignment and status methods exist. Cross-course completeness has not been verified.
FR-05 Chronological assignment list	UC-02	AssignmentOrganizer; Assignment	Agent orchestrator; assignment tools; LMS adapter	ADR-03	Gap: no deterministic cross-course sort with undated items last; model-generated ordering is not acceptance evidence.
FR-06 Due within seven days	UC-03	Assignment, Course	Agent orchestrator; assignment tools; LMS adapter	ADR-03	Gap: no dedicated seven-day filter or boundary tests; dates currently interpreted by model. UC-03 is listed but not one of Section 4's expanded cases.
FR-07 Understandable retrieval errors	UC-01–UC-04	AssistantInterface; AssignmentOrganizer; Generated answer	LMS adapter; orchestrator; FastAPI; chat panel	QAS-03; ADR-01	Partial: adapter exceptions become tool errors; API catches model-service failures. No 20-outage test or proof against misleading partial results.

Requirement	Use case(s)	Analysis objects / concepts	Design components	QAS / ADR	Coverage and verification gap
FR-08 Five course-based practice questions	UC-04	Course module / Page; Generated answer	Chat panel; orchestrator; read-only tools; Claude	ADR-03	Partial: tutoring prompt permits quizzes; complete five-question workflow and acceptance test unverified.
QR-01 95% within 5 s, 20 concurrent	UC-01–UC-04 (shared chat path); S-01 samples ordinary questions	Student question / Chat session; tool request	FastAPI; orchestrator; LMS adapter; Claude	QAS-01; ADR-01, ADR-03	Unverified in this contribution: no 100-request measurements inspected. See John's Section 8.
QR-02 9/10 first-time users within 60 s	UC-02, UC-03	AssistantInterface; AssignmentOrganizer	Moodle launcher; chat panel	ADR-04; no dedicated QAS in Section 6.1	Unverified: no 10-student usability study, completion counts or timing evidence.
QR-03 Deny all 20 unauthorized attempts	UC-01–UC-04	Student / Enrollment; Chat session	FastAPI identity boundary; LMS adapter; Moodle REST	QAS-02; ADR-05	Gap: chat endpoint has no per-student authentication. Session IDs are not identity credentials. Do not claim authorization compliance.
QR-04 95/100 factually correct	UC-01, UC-04; factual assignment answers in UC-02/03	Generated answer; Course / Page / Assignment	Orchestrator; tools; LMS adapter; Claude	ADR-03; no dedicated QAS in Section 6.1	Unverified: no predefined 100-question evaluation and instructor-reviewed answer key/results.

Requirement	Use case(s)	Analysis objects / concepts	Design components	QAS / ADR	Coverage and verification gap
CON-01 Self-hosted Moodle 5.3	UC-01–UC-04	Course, Student, Assignment, Submission; Page	LMS adapter; Moodle REST; Moodle/PostgreSQL deployment	ADR-02, ADR-05	Design/code support: Docker deployment and Moodle adapter exist. Mock-LMS S-01 would not verify live integration.
CON-02 Course resource limits	All use cases; project-wide	No single domain class	Layered FastAPI/agent/adapter deployment	ADR-01, ADR-02	Architectural choice limits service complexity; actual hosting, inference cost and resource feasibility remain unverified.

9.2 Gap analysis and actions

Gap / affected requirements	Evidence and consequence	Planned owner / closure evidence
Performance, QR-01 / QAS-01	Sequential model/tool calls can accumulate latency. No measured S-01 results were available to this contribution; latency feasibility remains open.	John now owns the Section 8 experiment: publish raw results, configuration and summary. Zaid: reconcile this matrix with John's evidence. Ahmad and Daniel: evaluate P3 optimizations and ADR consequences from the results.
Per-student identity, FR-02 / QR-03	All chats share a configured test token. Moodle token scoping does not authenticate the browser caller.	John: P3 LTI 1.3 plus a per-student credential/authorization design; record 20/20 denial tests before real student use.
Grounding and references, FR-03 / QR-04 / UC-01	Retrieval tools exist, but references and factual correctness are not guaranteed by the prompt.	Erik: source references; Omor: 100-question answer-key evaluation reaching 95 correct responses. Also run FR-03's 8-question acceptance check.
Assignment organization, FR-04–FR-06	Per-course retrieval does not establish complete cross-course ordering or correct seven-day boundaries.	Ahmad: aggregate across enrolled courses, sort with undated assignments last, test inside/outside the seven-day interval and exact boundary dates.

Gap / affected requirements	Evidence and consequence	Planned owner / closure evidence
Retrieval failure, FR-07 / QAS-03	Tool errors may be rephrased by the model; Moodle calls have 30-second timeouts while QAS-03 targets a visible response within 10 seconds. No outage test establishes compliance.	Zaid: P3 failure handling; run the 20-failure scenario, measure 19/20 within 10 seconds, and verify no fabricated data or leaked credentials.
Practice workflow, FR-08 / UC-04	General quiz capability does not verify five grounded questions or all alternate flows.	Zaid: P3 practice mode and acceptance cases against selected-course material.
Usability, QR-02	Embedded UI exists; no first-time user evidence.	Daniel: accessibility/UI work; arrange the defined 10-student study and record task times and assistance.
Cross-author analysis / ADR baseline	Section 5.1 and UC-01 interaction model unavailable; UC-02 and rationale still on separate PRs. Exact class and decision consistency cannot yet be certified.	Zaid: reconcile this matrix after models merge; Omor: K1–K6 cross-checks; Daniel/Ahmad: confirm ADR and component mappings.
Feasibility and stakeholder evidence	Section 2 explicitly records that survey evidence is unavailable; neither this matrix nor S-01 supplies student survey or hosting-cost evidence.	Zaid: collect and report actual aggregate survey data; John: ensure final assembly and contribution/AI-use records reflect remaining gaps.

No requirement is marked accepted solely because a trace exists. The matrix provides design coverage; acceptance requires the corresponding test evidence. This contribution changes documentation only; John's spike is separate, and listed production fixes remain P3 work.

AI assistance: Codex helped inspect the linked sources and draft the mappings and gaps.

10 Project Plan (updated; was P1 section 4)

10.1 Roles and working agreement

Roles and the decision rule are unchanged since v0.5: John Bramlet (Team Lead), Ahmad Zghier (System Architect & Technical Requirements Lead), Zaid Abusamra (Requirements and Feasibility Analyst), Omor Igiehon (Quality Assurance Tester), Daniel Habib (Product Manager / UI Engineer), Erik Sandoval (Requirements Engineer & Use Case Analyst). Disagreements are discussed in the team chat or a meeting; if still split, a majority vote decides and the team leader breaks a tie and records the decision in the repository.

What changed for P2: each member owns named sections (recorded in 0.1) and commits them through a pull request that another member reviews, using a fixed review chain (John reviews Erik, Erik reviews Omor, Omor reviews Daniel, Daniel reviews Ahmad, Ahmad reviews Zaid, Zaid reviews John). Where an assigned reviewer was unavailable, another member reviewed instead.

10.2 Plan to P3

Milestone	Date	Owner	Done when
Per-student identity: Moodle launches the agent through LTI 1.3 and the agent reads with that student's own access	Nov 22, 2026	John Bramlet	Two test students each see only their own grades; 20 negative tests disclose nothing (QAS-02)
Source references in answers	Nov 22, 2026	Erik Sandoval	Each answer to a course question names the page or assignment it came from (FR-03, UC-01)
Accuracy evaluation set and scoring script	Nov 22, 2026	Omor Igiehon	100 questions with an answer key in the repository; script reports accuracy against QR-04
Chat panel accessibility and CS 3354 sample content in Moodle	Nov 22, 2026	Daniel Habib	Panel usable by keyboard and screen reader at phone width; CS 3354 content loaded (R3)
Cross-course "due this week" tool with the seven-day flag; latency changes from S-01	Nov 22, 2026	Ahmad Zghier	FR-04 to FR-06 acceptance tests pass across two courses; S-01 rerun reported against QAS-01
Practice-question mode and LMS-failure messages	Nov 22, 2026	Zaid Abusamra	FR-08 and FR-07 acceptance tests pass; QAS-03 outage test run
Detailed design (design class and sequence diagrams), test plan, P3 document	P3 due date on Canvas	All, assembled by John Bramlet	P3 submitted with tag `p3`

10.3 Risks (updated)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R1 Setting up open-source Canvas takes longer than planned	Closed: replaced by self-hosted Moodle, running with a seeded course (CH-02, ADR-02)	n/a	None needed
R2 The LLM gives answers not grounded in course material (FR-03)	Open	Medium / High	System prompt limits answers to retrieved material; accuracy evaluation set in P3 (Omor)

Risk	Status since P1	Likelihood / impact	Mitigation and owner
R3 Sample course content too thin to test real scenarios	Reduced: CS101 seeded with pages, four assignments in different states, an announcement and a test student	Medium / Medium	Load CS 3354 content in P3 (Daniel)
R4 LLM API costs exceed what the team can pay for testing	Open	Medium / Medium	Measure cost per question during S-01 and the evaluation runs; lower effort or use a smaller model where quality holds (Ahmad)
R5 A team member becomes unavailable	Realized in P2 for one member's sections	Medium / Medium	Sections reassigned or recorded as not done in 0.1; review chain adjusted (John)
R6 (new) Several model turns per question make QR-01 (95% under 5 s, 20 users) unattainable	Open; created by ADR-03	High / High	Spike S-01 measures it; options are lower effort, fewer tool calls, streaming the first words, or revising QR-01 with evidence (Ahmad, Zaid)
R7 (new) The prototype uses one test student's token, so anyone who reaches the chat server sees that student's data	Open; created by ADR-04 and ADR-05	High / High	Test data only until LTI 1.3 per-student identity lands in P3 (John)
R8 (new) Student questions and course context leave the team's host for the external AI provider (FERPA)	Open; trust boundary in 6.4	Medium / High	Send only what a question needs (ADR-03); privacy review before any real student data (John)

10.4 Repository

URL: https://github.com/azghier1/Ai-Agent (public).

Layout: `/docs/p2` holds this PDF and the source of each section in `/docs/p2/sections`; `/docs/p2/diagrams` holds every diagram's source file and exported image; `/spikes` holds the architecture spike with its README and measurements; the prototype code is at the repository root (`agent.py`, `lms.py`, `server.py`, `web/`, `docker-compose.yml`, `moodle/`).

Rules: `main` holds only reviewed work. Each change is made on its own branch named `type/short-description` and merged through a pull request that another member approves. Commit messages start with the section or requirement they change (for example `6.3: add component view`). Credentials stay in a local `.env` file that is excluded from the repository.

The tag `p2` points at the commit that contains this PDF.

10.5 Use of AI tools in P2

Member	Tool	What it was used for	What you checked or changed
John Bramlet	Claude (Claude Code, Claude Opus 5.5)	Wrote and debugged the prototype (agent, Moodle adapter, chat server, chat bubble), set up Moodle in Docker and the sample-course seed, drafted the onboarding guide and work plan, and drafted sections 0, 1 and 10 to 12. Commits `b5c18f7`, `f019be3`, `c5ee6e5`, `347886c`, `9b3bad1`, `4db0b68`, `124579b`, `b908084` and the PR #6 and PR #8 commits carry its co-author line. Also ran the S-01 spike with Zaid Abusamra	Ran the prototype against Moodle and checked every tool's output against the course; checked each section against the P2 template, the P1 document and the repository history; checked every hash in 0.1 exists
Zaid Abusamra	Codex (OpenAI)	Inspecting repository sources and drafting sections 2 and 9 (PR #7)	Checked coverage against sections 3, 4, 5.2, 6 and 7, verified cited file paths and commits; Ahmad reviewed the PR
Daniel Habib	Claude (Claude Opus 5.5)	Drafting ADR-01 to ADR-05, the trade-off summary and the UC-02 scenario table and sequence diagram. Commits in PR #2 and PR #3 carry its co-author line	Aligned UC-02 with Erik's expanded steps in 4.2 and the ADRs with Ahmad's QAS IDs

11 Open Questions (updated; was P1 section 5)

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q1	Which LMS will the agent integrate with?	P1	Team, before P2	Closed: self-hosted Moodle 5.3 (CH-02, ADR-02)	n/a
Q2	What tasks should the agent perform?	P1	Team, before P2	Closed: answer questions about a student's courses, assignments, grades, syllabus, pages and announcements, and tutor without doing graded work; practice questions in P3 (FR-08)	n/a
Q3	What LMS data can the agent access?	P1	Team, before P2	Closed: only what the student can see, read with the student's own credential (ADR-05)	n/a

ID	Question	Raised by	Owner and due date	Status or answer	Assumption meanwhile
Q4	Do students' stated needs match FR-01 to FR-08?	P1 feedback (C1)	Zaid Abusamra, Nov 15	Open: survey in 2.2	Needs as stated in 2.2
Q5	Can QR-01 be met with on-demand tool calls, or must QR-01 or ADR-03 change?	Analysis (R6)	Ahmad Zghier and Zaid Abusamra, Nov 22	Open: S-01 gives the first measurement	QR-01 stands as written
Q6	How does the agent obtain each student's own Moodle access after an LTI 1.3 launch?	Design (ADR-05)	John Bramlet, Nov 22	Open	One test student's token, test data only

12 Baseline v1.0 Declaration (was P1 section 6)

We declare this document, version 1.0, the baseline for P2. Sections 1 to 9 describe what we will build in P3. Any later change to them is recorded in section 0.2 with its reason.

Team member	Signature or initials	Date
John Bramlet	John Bramlet	Nov 8, 2026
Erik Sandoval	ES	11/08/2026
Omor Igiehon	Did not sign	
Daniel Habib	DH	11/08/2026
Ahmad Zghier	Not signed by deadline	
Zaid Abusamra	Zaid Abusamra	Nov 8, 2026